

Assignment 2: Model-free reinforcement learning

Carlo Dubbioso (s4416090) & Maria Mukhovykova (s4379055)

Group Number: 55

1 The ShortCut Environment

This assignment focuses on the implementation of four Reinforcement Learning Agents and comparing them to master how each one of them performs and which maximises the cumulative reward the best. These agents are set in a custom environment, “ShortCut” environment, that consists of a 12×12 grid in which the agent can move to adjacent squares within the grid (up, down, left, right). The goal of the agents is to reach the “goal square” (green square). When this is reached the episode ends and a new one begins. In each episode the agent starts in one of two squares (blue squares) with equal probability. The red squares in the grid correspond to the ‘cliffs’, if reached the agent returns to the starting position with a reward of -100. All other actions have a reward of -1. Which agent will perform best?

2 Q-Learning Agent

2.1 Methodology

The first agent we go through is the Q-Learning Agent by completing its class. Q-Learning is an off-policy, Temporal Difference (TD) control algorithm whose goal is to estimate the optimal action-value function “ $Q(s,a)$ ”. By being an off-policy algorithm, the agent maintains two distinct policies: a behavioural policy “ b ” and a target policy “ π ”. The behavioural policy consists in what the agent does while exploring. The target policy is, instead, what the agent is trying to learn.

The heart of the Q-Learning algorithm is in the “update()” function where its update equation lies:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [R_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

Where “ s ” stands for the state, “ a ” for the action at timestep “ t ”, “ α ” is the learning rate which controls how much the agent trust the new experience compared to what it already know, “ r ” is the immediate reward after taking a certain action, and “ γ ” is the discount factor which controls how much the agent cares about future rewards versus immediate ones. Half of the update function is between brackets “[]”. This represents the Temporal Difference (TD) error: the difference between what you would expect and what you actually get. This is why the “max” function is introduced, to select the action with the highest estimated Q-value in the next state s' , this allows the agent to learn the optimal policy regardless of which action it takes next.

For the “action_selection()” function we use the ϵ -greedy policy, approaching the exploration-exploitation tradeoff we, likewise, encountered in the multi-armed bandits assignment, which balances how much the agent should exploit its knowledge against randomly choosing the following action.

Lastly, we implement a “train()” loop function that trains the agent over “n_episodes” and returns a vector containing the cumulative reward per episode.

2.2 Greedy Policy Visualization

After running a single experiment with our newly implemented “train()” function for “n_episodes=10000”, to visualise the greedy policy, we end up with the following grid (Figure 1). The representation of this grid is given by the “render_greedy()” function defined in our environment class, which illustrates the path within the environment to reach the goal state. We notice that from the agent, when starting from the lower-left state, in cell [2,9], takes the shortcut, showing that even though the path is within a tunnel of cliffs, at the end of the 10000 episodes the agent was able to find the fastest path to reach the goal state from that starting position. The same goes for the higher-left starting state, which path is bordered by cliffs.

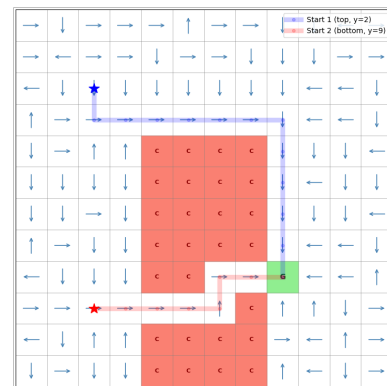


Figure 1: Q-Learning Greedy Policy

2.3 Learning Curve

To obtain a reliable estimate of the Q-Learning algorithm expected performance, we averaged the episode rewards over 100 repetitions of 1.000 episodes each, using the default hyperparameters $\alpha = 0.1$ and $\epsilon = 0.1$, because given the presence of stochasticity, introduced by the ϵ -greedy policy, a single run would be far too noisy to draw meaningful conclusions. The resulting curve shows that the agent begins with very negative average returns, reflecting the early phase where it has little knowledge of the environment and frequently falls off the cliff. As episodes increase, the curve rises steadily, indicating that the agent is accumulating knowledge and adjusting its Q-value estimates. Around episode 400, the curve begins to flatten slightly below 0. Nevertheless, even after averaging across 100 repetitions, the curve still has some degree of noise, this is because the agent continues to take random actions with probability $\epsilon = 0.1$, even after having found the correct path, and therefore walking off the cliff and producing dips in the reward signal.

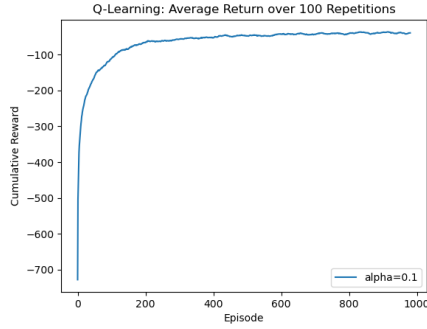


Figure 2: Q-Learning with $\alpha=0.1$, smoothed

2.4 Effect of Alpha

From comparing different values of α , we examine the effect that the learning rate has on our agent performance. We used $\alpha = [0.01, 0.1, 0.5, 0.9]$ over 1000 episodes across 100 repetitions. From the resulting plot (Figure 3), the most striking case is $\alpha = 0.01$: because each update only pushes the Q-values by 1% of the TD error, the agent accumulates knowledge extremely slowly. Even after 1000 episodes, its cumulative reward has not yet converged, still sitting around -200 , confirming that while a very low learning rate is theoretically stable, it is simply too slow to be practical within a limited training set. On the other hand, $\alpha = 0.9$ converges the fastest, reaching a stable reward close to 0 within the first few dozen episodes. The intermediate values $\alpha = 0.1$ and $\alpha = 0.5$ both converge quickly and stably, with $\alpha = 0.5$ converging slightly faster, and both flatten at a reward close to 0.

These results reflect the fundamental trade-off that α introduces: set it too low, and the agent is too slow to learn anything useful within a reasonable number of episodes; set it too high, and while convergence is fast here, the agent risks overwriting well-established Q-values with misleading estimates from noisy transitions.

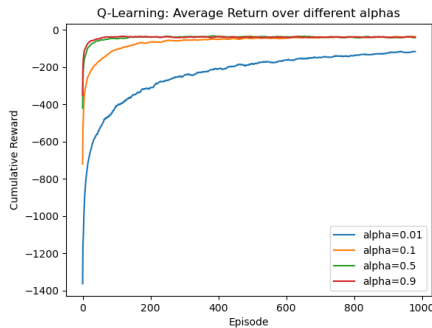


Figure 3: Q-Learning comparison over different alphas, smoothed

3 SARSA Agent

Our second agent we will be going through is the SARSA Agent. SARSA compared to the previous Q-Learning, is an on-policy Temporal Difference control algorithm: it learns the value of the policy it is executing, including the exploratory moves made by the ϵ -greedy behaviour policy. Unlike Q-Learning, which exploits the best possible next action regardless of what the agent actually does, SARSA uses the action that the agent will take next, selected by the same ϵ -greedy policy. Its update rule is therefore:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot Q(s', a') - Q(s, a)]$$

where each component carries the same meaning as in Q-Learning, with the only difference being the absence of the max operator: $Q(S_{t+1}, A_{t+1})$ replaces $\max_a Q(s_{t+1}, a)$, making the agent accountable for its own exploratory behaviour.

3.1 Greedy Policy

The most striking difference from Q-Learning is visible in the greedy paths (Figure 4). Q-Learning converges to the shortest path because it learns as if it always acts greedily in the future. SARSA, on the other hand, discovers a longer but safer route that navigates around the cliff, staying at least one square (step) distance from it at all times. This is a direct consequence of on-policy learning: during training, the agent occasionally takes random actions near the cliff and falls in, receiving a -100 penalty. SARSA encodes this risk into its Q-values, making cliff-adjacent states appear less attractive even under the greedy policy. This mirrors the results observed in the Cliff Walking experiment by Sutton & Barto, who observe that Q-learning learns values for the optimal policy travelling right along the cliff edge, ϵ -greedy action selection, whereas SARSA takes its own exploration into account and learns the longer but safer path instead[1].

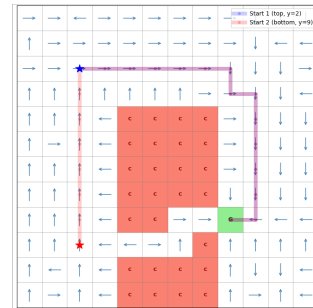
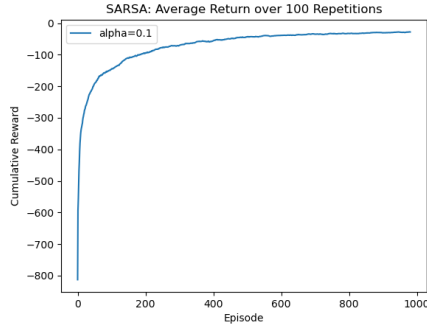


Figure 4: SARSA Greedy Policy

3.2 Learning Curve

The average learning curve over 100 repetitions of 1000 episodes with $\alpha = 0.1$ (Figure 5) shows a shape almost identical to the Q-Learning: the agent starts with very negative cumulative rewards due to frequent cliff falls in the early episodes, then improves steadily before flattening around episode 400-500, retaining some noise throughout due to continued random exploration.

Figure 5: SARSA Agent with $\alpha=0.1$, smoothed

3.3 Effect of Alpha

The comparison across $\alpha = [0.01, 0.1, 0.5, 0.9]$ (Figure 6) mirrors the pattern seen for the Q-Learning. The $\alpha = 0.01$ curve is by far the slowest, failing to converge within 1000 episodes and still sitting around -200 at the end. Higher values of α converge much faster, with $\alpha = 0.5$ and $\alpha = 0.9$ reaching a stable flattening close to 0 within the first hundred episodes. Interestingly, $\alpha = 0.9$ shows a slight decline in the final episodes, suggesting that such an aggressive learning rate may cause more instability once the agent is near convergence, as individual noisy transitions can overwrite well-established Q-values. The overall structure of these results confirms that α plays the same role in SARSA as in Q-Learning and the same trade-off between speed and stability.

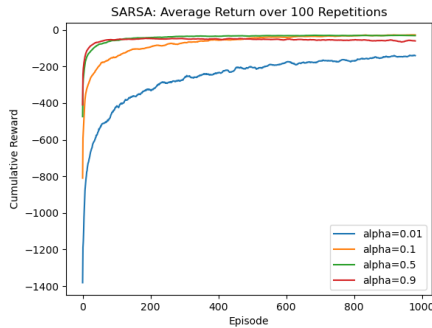


Figure 6: SARSA Agent comparison over different alphas, smoothed

4 Stormy Weather

After testing both agents on a deterministic environment, we train them on the WindyShortcutEnvironment, in which after each action there is a 50% probability of the agent being pushed one cell downward by the wind. We train Q-Learning and SARSA for 10,000 episodes with $\alpha = \epsilon = 0.1$ and $\gamma = 1.0$, and visualise the paths that the greedy policy of each agent takes from the two possible starting positions.

4.1 Results

Q-Learning. Comparing the normal and stormy Q-Learning plots (Figure 7a), the most visible change in the arrow field is that many cells that previously pointed rightward or downward now point upward, reflecting the need to counteract the downward drift of the wind. The path from Start 2 notably differs from the deterministic case; rather than taking the shortcut close to the cliff, the agent now avoids it entirely by skirting around the left side, as the wind makes accidentally falling into the cliff too likely for that route to be worth taking. The path from Start 1 is largely similar to the non-windy case, though it maintains a slightly larger margin from the cliff, suggesting the agent has learned to keep a small safety buffer against unexpected downward shifts.

SARSA. The stormy SARSA plot (Figure 7b) shows a more dramatic adaptation than Q-Learning. The arrow field contains substantially more upward-pointing arrows, particularly in the columns to the left of the cliff, compared to the non-windy case. The greedy paths reflect this: both starting positions take routes that climb higher than necessary before moving rightward, building a protection against being pushed downward into the cliff by the wind. As already observed in the deterministic environment, SARSA refuses to skirt the cliff even when the probability of falling into it is low, a tendency that is strongly heightened in the stormy environment. This is a direct consequence of SARSA's on-policy nature: during training, the agent repeatedly experienced being pushed toward the cliff by the wind, which raised the perceived cost of cliff-adjacent states and nudged the learned policy toward safer, higher paths.

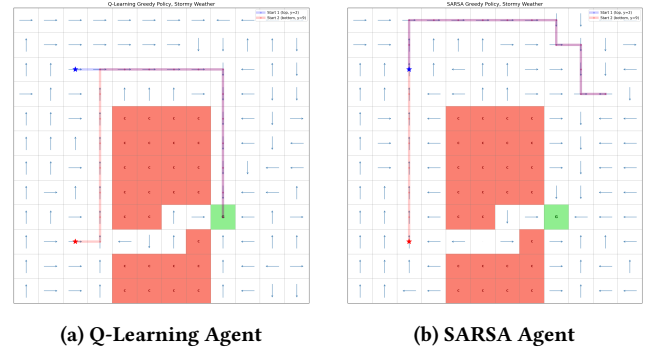


Figure 7: Results comparison

4.2 Interpretation

The core difference between the two algorithms in the windy environment mirrors and amplifies the difference seen in the original deterministic environment. Q-Learning learns the value of the best action from each state under the assumption of future optimality, which makes it less sensitive to the stochastic wind. SARSA, instead, by being on-policy, updates the Q-values based on the action it actually takes next, including the consequences of being randomly pushed downwards by the wind. This indicates that SARSA analyses the wind's danger more in depth, producing a policy that sacrifices path length for robustness.

If the wind would have come from a different direction, pushing the agent upwards rather than downwards, both policies would

adapt their paths accordingly, with arrows pointing more downward throughout the grid. More interestingly, upward wind would reduce the danger of the cliff, since the cliff occupies central-lower rows, potentially allowing both agents to take more direct routes. In this case the difference between SARSA and Q-Learning would shrink, since the stochastic disturbance would push the agent away from the cliff rather than toward it.

5 Expected SARSA

5.1 Methodology

The idea behind Expected SARSA algorithm is that it updates the action value function $Q(s, a)$ based on the expected value of the next state under the current policy instead of sampling a single next action as in basic SARSA. The equation for the update rule [1] is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \sum_{a'} \pi(a'|s') \cdot Q(s', a') - Q(s, a)]$$

Using ϵ -greedy policy, the probability of choosing greedy action is $1 - \epsilon + \epsilon/|A|$, and other actions get probability $\epsilon/|A|$. In Expected SARSA we compute the expected value over all possible actions at the next state, what makes the update deterministic based on current Q-values and reduces variance compared to standard SARSA. Similarly to Q-Learning, Expected SARSA bootstraps from fixed policy target, however, the algorithm does not assume perfect greedy behaviour and the target is the policy-weighted expectation. This ensures smoother updates and more consistent behaviour. The agent was trained with $\epsilon = 0.1$, $\gamma = 1.0$, and different values of α .

5.2 Results

In Figure 8 we can see the visualisation of the greedy policy, it shows that, like SARSA, Expected SARSA converges to the safer path along the upper part of the grid and stays away from the edges. This shows the difference from Q-Learning, which mainly takes optimal path because it bootstraps from the maximum Q-value and ignores the risk of ϵ -greedy exploration during training. Meanwhile, Expected SARSA uses the policy-weighted expectation where cliff-adjacent cells get small but non-zero probability of falling and update rule in this case pushes the learning values downward and moving the greedy path to the safer way.

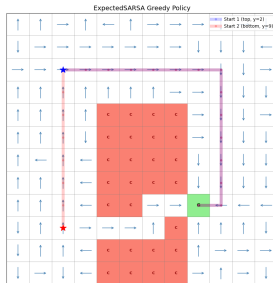


Figure 8: Expected SARSA Greedy Policy

5.3 Interpretation

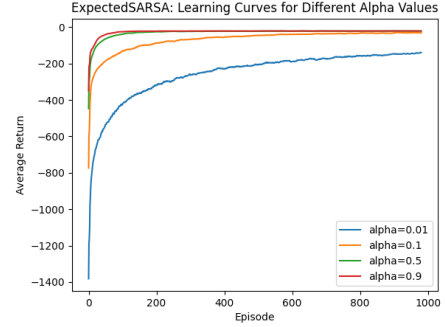


Figure 9: Learning Curve for Expected SARSA with $\alpha \in \{0.01, 0.1, 0.5, 0.9\}$

Observing the learning curve, we can see a trade-off between speed and stability with respect to α . Small values like $\alpha = 0.01$ show steady improvement, and the curve is still not converging over 1000 episodes. When α values are around 0.1 then stability can be achieved quicker and balance is better. Meanwhile, larger α (0.5 or 0.9) coverages faster in the beginning but also presents more variance, and the updates go back and forth.

In contrast to Q-Learning and SARSA, in Expected SARSA we can observe a lower variance over repetitions with equivalent α values. It happens because expected target is less noise for both sampled next action target of SARSA and max target of Q-Learning. In the standard environment Expected SARSA and SARSA converge similarly - both choosing the safer path; while Q-Learning gets higher asymptotic return as it is more sensitive during exploration stage in training.

6 n-step SARSA

6.1 Methodology

N-step SARSA uses a return that looks n steps into the future before bootstrapping, which all together generalises one-step SARSA algorithm. The n -step return looks as:

$$G_{t:t+n} = r_{t+1} + \gamma \cdot r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n Q(s_{t+n}, a_{t+n})$$

and the update applies this return to the state-action pair n steps in the past:

$$Q(s_\tau, a_\tau) \leftarrow Q(s_\tau, a_\tau) + \alpha [G_{t:t+n} - Q(s_\tau, a_\tau)]$$

where $\tau = t - n + 1$ is the time step to update.

In practice, updates are delayed because after the first n steps of an episode, the agent starts to update the earlier state-action pairs in the buffer, and when the episode finishes, any left buffered updates are flushed and terminal rewards are used without bootstrapping.

6.2 Results

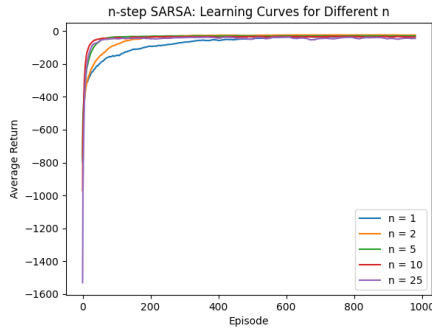


Figure 10: Learning Curve for n -step SARSA with $n \in \{1, 2, 5, 10, 25\}$

The agent was trained with $\epsilon = 0.1$, $\gamma = 1.0$, $\alpha = 0.1$ and $n \in \{1, 2, 5, 10, 25\}$. The learning curve, that was obtained over 100 repetitions of 1000 episodes, shows non-monotonous relationship between n and learning efficiency. With $n = 1$, the same as in one-step SARSA, the agent improves consistently but slowly, this happens because each update includes only one reward signal before going back to the bootstrap estimate $Q(s_{t+1}, a_{t+1})$. When n goes to 2 and 5, the curves rise sharply and reach a higher average return in 1000 episodes. The improvement happens because in a single update the longer returns can generate future rewards and send that information back through episodes. However, the learning curves of

$n = 5, 10, 25$ are nearly the same through training, showing that adding more steps does not provide more information to the agent. Nonetheless, the performance across all n values is quite similar, suggesting that the choice of n affects early learning rather than quality of final policy.

6.3 Interpretation

The results confirm that multi-step updates definitely speed up the learning process compared to standard SARSA. This is especially helpful in ShortCut environment, where agent needs to take many steps before getting to the goal. The improvement is visible already when we change n value from 1 to 2 or 5, which confirms that one-step update is too slow.

As mentioned previously, larger n is not very beneficial, and the reason behind it is the bias-variance trade-off. When $n = 1$ everything relies on the bootstrap estimate, which creates bias in an early training, however, when n grows, the bias reduces as more real rewards are used. Moreover, the variance increases because the update stage depends on longterm action sequence. In case if we are going to set $n \rightarrow \infty$, the algorithm becomes full Monte Carlo, where it uses only real rewards and forgets completely about bootstrapping. Although the bias disappears, the waiting of the episode's end to make any updates resulting in slow learning and very long episodes. From the obtained results the greatest advantage was captured by $n = 5$, suggesting a good balance between bias and variance.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.